Politecnico di Milano

Ingegneria del Software – a.a. 2008/09

Appello del 09 Settembre 2009

Cognome
Nome
Matricola
Sezione (segnarne una) Baresi ☐, Ghezzi ☐, San Pietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h15m.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 6)

Si consideri il seguente metodo statico:

public static String[] permutazioni (String s)

Il metodo riceve come parametro una stringa non vuota di caratteri tutti diversi tra loro. Il metodo restituisce l'insieme di tutte e sole le permutazioni dei caratteri che compongono la stringa in ingresso, questa compresa, senza ripetizioni e in ordine casuale.

Ad esempio, se fosse "ABC", il metodo dovrebbe restituire: {"ABC", "ACB", "BAC", "BCA", "CAB", "CBA"}, includendo quindi tutte le 3!=6 permutazioni di "ABC".

Specificare **permutazioni(...)** in JML.

Suggerimento: Si consiglia di specificare dapprima un metodo puro della classe String, denominato public /*@pure@*/ boolean permut(String x), che restituisce true se x e' una permutazione di this, false in ogni altro caso. Utilizzare poi il metodo permut() nella specifica di permutazioni(..).
Per accedere ai singoli caratteri di una stringa, si può utilizzare il metodo della classe String "char charAt(int index)", che restituisce il carattere in posizione index di this, con index che va da 0 a this.length()-1.

SOLUZIONE:
permut può essere definita in modo generale, anche per casi con caratteri ripetuti, come segue, segnalando che il numero di occorrenze di ogni carattere in x è lo stesso del numero di occorrenze di quel carattere in this:

```
//@ensures \result == (x!= null && x.size()==this.() &&
//@ (\forall int i; 0<=i && i<x.length();
//@     (num_of int j; 0<=j && j<x. length (); x.charAt(j)==x.charAt(i))
//@     ==
//@     (num_of int k; 0<=j && j<this.size(); this.charAt(k)==x.charAt(i)))
public /*@pure@*/ boolean permut(String  x)
```

Permutazioni può a questo punto essere definito dicendo che ogni elemento in \result è una permutazione di s, che ogni elemento di \result è distinto, e che in tutto \result ha esattamente s.length()! elementi:

```
//@requires x!=null &&
//@ (\forall int i; 0<=i && i<s.length(); (num_of int j; 0<=j && j<s.length();
//@     s.charAt(i)==s.charAt(j))==1) //no ripetizioni in s
//@ensures (\forall int i; 0<=i && i<result.length;
//@     s.permut(result[i])) &&
//@     (num_of int j; 0<=j && j<result.length; \result[i].equals(result[j])==1) &&
//@ \result.length== Math.fact(s.length())
public static String[] permutazioni (String s)
```

Esercizio 2 (punti 14)

Si consideri un'applicazione java di supporto al gestore di un social network. L'applicazione include una classe Java "Membro", che descrive un partecipante al network.
La classe Membro ha, fra gli altri, i seguenti metodi modificatori, correlati alla possibilità per un membro di diventare "amico" di altri membri:

1. public void invita (Membro m)
   il cui effetto è di invitare un altro membro del social network a diventare un amico del membro this, purche' non si cerchi di invitare se stessi oppure un membro che in passato abbia già risposto positivamente all'invito.
2. public void accettaInvito (Membro m)
   il cui effetto è di accettare l'invito di un altro membro, purché questi abbia effettivamente invitato il membro this, rendendo quindi "amici" this e m.
3. public void declinaInvito (Membro m)
   il cui effetto è di rifiutare l'invito di un altro membro a this, rimuovendo il relativo invito di m.

Si osservi che, in base a questi metodi, le possibili relazioni fra due membri A e B sono solo le seguenti:

- A e B sono amici (ossia uno ha invitato l'altro, che ha accettato);
- A e B non sono amici, ma uno ha invitato l'altro, che non ha ancora risposto;
- A e B non sono amici, e non vi sono inviti pendent (perché nessuno dei due ha mai invitato l'altro oppure perché uno ha invitato l'altro che però ha rifiutato).

a) *Si specifichino i precedenti metodi 1 ,2 e 3 della classe Membro in JML*. Si ipotizzi a tale scopo che Membro fornisca anche i seguenti osservatori puri:

```
//restituisce l'elenco dei membri invitati da this, che non hanno risposto:
public   /*@ pure */ ArrayList<Membro>  invitati()

//restituisce l'elenco degli amici di this:
public  /*@ pure */ ArrayList<Membro>  amici()
```

e il seguente costruttore, di cui si presenta una specifica parziale, "depurata" dei vari argomenti come nome, email, ecc., che sono per semplicità ignorati in questa specifica:

```
//@ensures amici().size()+invitati().size()==0 &&…altre condizioni…;
public Membro(…)
```

SOLUZIONE:
Si utilizzano per semplicità i metodi contains e containsAll delle Collection (che verificano rispettivamente l'appartenza di un oggetto e l'inclusione di una collezione nell'ArrayList this).
Al solo scopo di abbreviare le specifiche, definiamo un ulteriore metodo statico puro:

```
//@requires a1!= null && a2!=null && x!=null
//@ensures \result == (a1.containsAll(a2) && !a2.contains(x) &&
//@ a1.contains(x) && a1.size() == 1+a2.size()
public /*@pure@*/ static boolean differenza(ArrayList<T> a1, ArrayList<T> a2, <T> x)
```

il cui significato è quindi che a1 contiene tutti gli elementi di a2 (non importa in che ordine) e inoltre il solo x, ossia x è l'unico elemento di differenza fra a1 e a2.

```
//@ requires !m == null && !m==this && !amici().contains(m) &&
//@ //non ci sono inviti pendenti fra m e this:
//@!invitati().contains(m) &&!m.invitati().contains(this);
//@ ensures differenza(invitati(),\old(invitati()), m) && //m è nuovo invitato
//@ amici().equals(\old(amici()));
//@assignable this.*;//solo this è modificato, m è inalterato.
public  void invita (Membro m)

//@ requires !m == null && m.invitati().contains(this);
//@ ensures differenza(\old(m.invitati()), m.invitati, this) &&
//@ differenza(amici(),\old(amici()), m) &&
/@  differenza(m.amici(),\old(m.amici()),this) &&
/@ invitati().equals(\old(invitati()));
//@ assignable this.*, m.*;//sia this che m sono modificati.
public void accettaInvito (Membro m)

//@ requires !m == null &&  && m.invitati().contains(this);
//@ ensures differenza(old(m.invitati()),m.invitati(),this)
//@ m.amici().equals(\old(m.amici()));
//@ assignable m.*;//solo m è modificato, this è inalterato.
public void  declinaInvito (Membro m)
```

b) Si considerino le seguenti proprietà, che sono parte del contratto di Membro:

I) "Un membro non può invitare un amico"

II) "La relazione di amicizia fra membri è simmetrica, ma non riflessiva".

III) "Se due membri A e B sono amici, allora in passato A ha invitato B oppure B ha invitato A, e l'invito è stato accettato dall'altro membro"

b.1) Di che tipo di proprietà si tratta?

SOLUZIONE: Le prime due sono invarianti pubblici, la terza è una proprietà evolutiva. La prima potrebbe in realtà anche essere considerata una proprietà evolutiva, in quanto predica su un cambiamento di stato; tuttavia, come nel punto successivo, può essere facilmente espressa con un invariante pubblico.

b.2) Descriverle in JML, se possibile. Nel caso non sia possibile farlo, spiegare come mai accade questo.

SOLUZIONE: I e II) sono:

I) public invariant (\forall Membro m; amici().contains(m); !invitati().contains(m));

II) public invariant (\forall Membro m; amici().contains(m); m.amici().contains(this)) && !this.contains(this);

III) Non può direttamente essere espressa in JML, salvo che ricorrendo a qualche "trucco" (come inserire la proprietà come postcondizione dei metodi pubblici modificatori).

b.3) Le proprietà sono verificate dalla specifica data?

Se la soluzione e' quella data SI, in quanto:

I) L'invito ad un amico comporta la violazione della requires di invita(..);

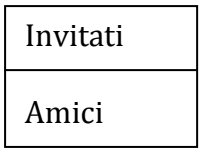
II) La postcondizione di accettaInvito include "amici().contains(m) && m.amici().contains(this)" garantendo la simmetria. L'invito a se stessi comporta la violazione della requires di invita(..);

III) Ogni membro all'inizio (v. il costruttore) non ha né amici né inviti, e l'unico modo disponibile per definire amici è tramite accettaInvito, che richiede che in precedenza sia stato inserito un invito per this.

c) Si consideri la seguente implementazione parziale di Membro:

```
//rep:
private ArrayList<Membro> altriMembri;
private int inizioInvitati;
public Membro(…)//vari parametri ignorati qui{
        altriMembri= new ArrayList<Membro>;
        inizioInvitati=0;
}
```

L'ArrayList altriMembri è suddiviso in DUE porzioni. La prima porzione, nelle posizioni da 0 a inizioInvitati-1 , include tutti e soli gli amici di this; la seconda porzione, nelle posizione da inizioInvitati alla fine di altriMembri (ossia da inizio Invitati ad altriMembri.size()-1) include tutti e soli gli invitati da this che non hanno ancora risposto. Se inizioInvitati vale altriMembri.size(), allora non vi sono invitati, mentre se inizioInvitati vale 0 allora non vi sono amici.



c1) Scrivere in JML l'invariante di rappresentazione della classe Membro.

SOLUZIONE:

```
private invariant altriMembri!=null && 0<= inizioInvitati && inizioInvitati<=altriMembri.size() &&
 (\forall int i; 0<= i && i<altriMembri.size(); altriMembri.get(i) !=null)
```

c2) Descrivere, tramite un private invariant di JML, che metta in relazione i metodi osservatori puri e le parti private, una possibile funzione di astrazione della classe Membro, ossia una formula JML che descriva come un oggetto concreto, ossia definito dal rep, corrisponda ad un oggetto astratto, ossia definito dagli osservatori.

SOLUZIONE:

```
private invariant
altriMembri.size()=amici().size()+invitati().size() &&
(\forall int i; 0<= i && i< altriMembri.size();
  altriMembri.get(i).equals(  i<inizioInvitati  ? amici().get(i) :
                                  invitati().get(i))) ;
```

c3) Implementare il metodo declinaInvito.

SOLUZIONE:

```
public  void declinaInvito(Membro m) {
(for int i =m.inizioInvitati; i<m.altriMembri.size(); i++)
   if  (m.altriMembri.get(i).equals(this))  {
          //ultimo elemento copiato in pos. i:
          m.altriMembri.set(i, m.altriMembri.lastElement());
          //rimuove ultimo elemento:
          m.altriMembri.remove(m.altriMembri.size()-1);
          return;
   }
}
```

d) Si consideri una nuova classe MembroPrivilegiato, erede della classe Membro. Un membro privilegiato del social network può seguire la consueta procedura ad inviti per diventare amico di un altro membro. Tuttavia, ha la facoltà di potere diventare amico di un membro non privilegiato anche se questi non l'ha invitato o non ha accettato un suo invito.
A tale scopo, la classe fornisce un metodo public void diventa Amico(Membro m),

1) *Specificare tale metodo*, inserendo anche eventuali precondizioni o eccezioni se ritenute opportune:

SOLUZIONE:

```
//@requires m!= null && !m instanceof MembroPrivilegiato;
//@ ensures (\forall Membro m1; m!=m1 && m1!=this;
 //@     (amici().contains(m1) <==>\old(amici().contains(m1))) &&
/@       (m.amici().contains(m1) <==>\old(amici().contains(m1)))) &&
//@ amici().contains(m) &&
//@ m.amici().contains(this) &&
//@ invitati().equals(\old(invitati()))&&
//@ m.invitati().equals(\old(m.invitati());
//@ assignable this.*, m.*;
```

public void diventa Amico(Membro m)

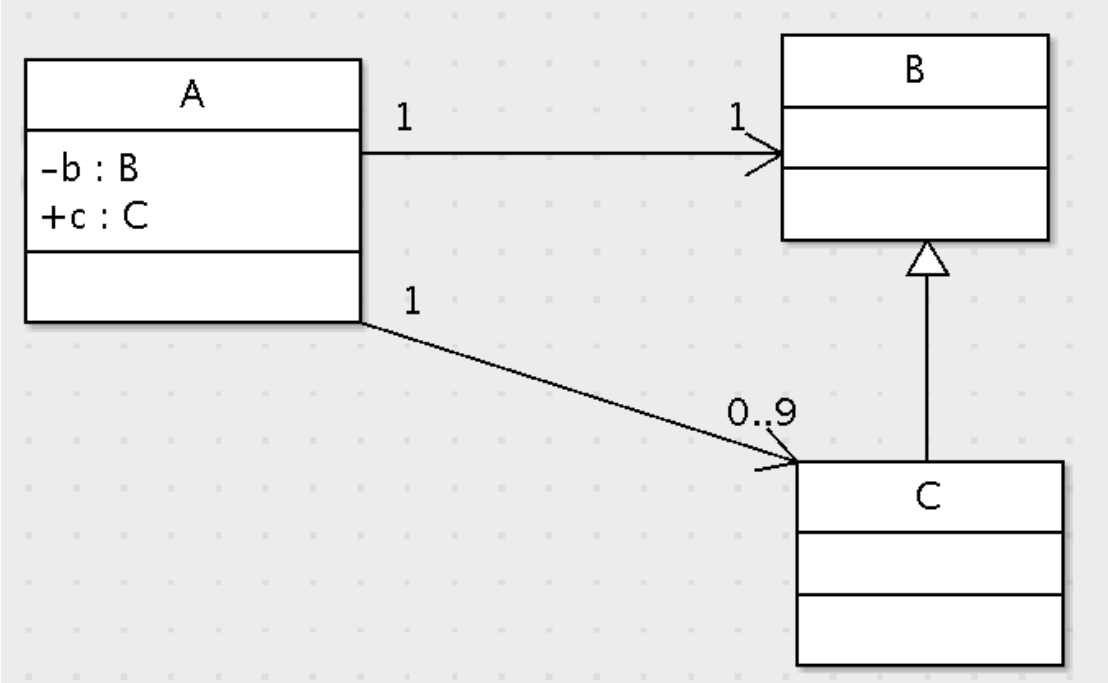
2) Commentare sinteticamente, ma efficacemente, sulla possibilità che MembroPrivilegiato rispetti, come erede di Membro, il principio di sostituzione di Liskov.

SOLUZIONE: le regole delle segnature e dei metodi sono verificate in quanto MembroPrivilegiato ha esattamente gli stessi metodi di Membro, con le stesse specifiche, oltre a un metodo addizionale. Tuttavia, non vale più la proprietà evolutiva B.III, che era da considerarsi parte integrante del contratto (anche se non esprimibile direttamente in JML). Pertanto , il principio di sostituzione NON è verificato.

Esercizio 3 (punti 5)

(A) Si consideri una classe A che ha come attributo pubblico un elemento di classe B e come attributo privato un array di 10 elementi di classe C. Si supponga anche che C erediti da B. Si disegni un diagramma UML che rappresenta questo frammento di programma.

Soluzione



(B) Si considerino i seguenti frammenti Java

```
interface X {
        void x1();
        float x2 (Yy);
}
class Y {
        public void y1() {...}
        public float y2 (X x) {...}
}
class Z implements X extends Y {
        public void y1(float a) {...}
        public float y2 (X x) {...}
        public void x1() {...}
        public float x2 (Y y) {...}
}
```

e le dichiarazioni seguenti: float i, j; X a, Y b; Z c;
A fianco di ciascuna delle seguenti istruzioni si scriva SI o NO a seconda che l'istruzione sia corretta o no dal punto di vista del "type checking" e si scriva una sintetica motivazione, in particolare specificando quale metodo viene chiamato.

a. x1( ); (SI, nella situazione attuale l'oggetto a cui a riferisce puo' solo essere un elemento di classe Z, per cui verra' chiamata x1 implementata da Z)

i = a. x2(c); (SI, v. sopra, si chiama la x2 di Z, cha come parametro si aspetta un Y, e Z e' sostituibile a Y)

a = c; (SI)

i = b. y2(a); (SI, si chiama la y2 di Y o se il tipo dinamico dell'oggetto dovesse essere Z, la y2 ridefinita in Z)

j = b. y2(c); (SI, c.s., essendo Z sostituibile a X)

c = a; NO

c = (Z) a; (SI, ma possibile errore a run time se il tipo concreto di a non dovesse essere Z)

c. y1( ); (SI, viene chiamata la y1 ereditata da Y)

c. y1(3.77); (SI, viene chiamata la y1 definita da Z, che genera un overloading di quella ereditata)